Scalable Chain of Thoughts via Elastic Reasoning

Yuhui Xu **Hanze Dong** **Lei Wang** **Doyen Sahoo** **Junnan Li** **Caiming Xiong**

Salesforce AI Research

Abstract

Large reasoning models (LRMs) have achieved remarkable progress on complex tasks by generating extended chains of thought (CoT). However, their uncontrolled output lengths pose significant challenges for real-world deployment, where inference-time budgets on tokens, latency, or compute are strictly constrained. We propose **Elastic Reasoning**, a novel framework for scalable chain of thoughts that explicitly separates reasoning into two phases—*thinking* and *solution*—with independently allocated budgets. At test time, Elastic Reasoning prioritizes the completeness of solution segments, significantly improving reliability under tight resource constraints. To train models that are robust to truncated thinking, we introduce a lightweight *budget-constrained rollout* strategy, integrated into GRPO, which teaches the model to reason adaptively when the thinking process is cut short and generalizes effectively to unseen budget constraints without additional training. Empirical results on mathematical (AIME, MATH500) and programming (LiveCodeBench, Codeforces) benchmarks demonstrate that Elastic Reasoning performs robustly under strict budget constraints, while incurring significantly lower training cost than baseline methods. Remarkably, our approach also produces more concise and efficient reasoning even in unconstrained settings. Our code has been made available at `https://github.com/SalesforceAIResearch/Elastic-Reasoning`.

1 Introduction

Large reasoning models (LRMs) [4, 25] have demonstrated remarkable performance on complex reasoning tasks by producing extended Chain-of-Thought (CoT) outputs, which facilitate effective problem-solving in domains such as mathematics and programming. Reinforcement learning (RL) techniques [29, 43, 27, 5, 30], have been employed to optimize these reasoning trajectories, enabling LRMs to generate longer, more informative chains. These RL-driven methods scale effectively across diverse benchmarks [44, 6, 21, 39, 20], yielding substantial gains in both solution accuracy and robustness; while they often incur significantly longer inference chains [4, 7, 41, 26, 38]. Notably, the length of the reasoning trajectory remains uncontrolled, making it difficult to allocate a fixed compute budget at inference time while maintaining a desired performance level.

Two primary lines of research have been proposed to address this challenge. The first, known as **Long2Short** [34, 14], seeks to reduce reasoning length through reinforcement learning with trajectory penalties or compression-aware fine-tuning, where the model is trained on shortened trajectories to preserve performance while minimizing inference cost. The second line of work focuses on **length control** [24, 1, 42]. S1 [24] introduces a simple mechanism that prompts the model to emit special tokens (e.g., "Wait", "Final Answer") to regulate reasoning length. However, this approach significantly degrades performance, as it overlooks the critical role of the solution segment. L1 [1] proposes a reinforcement learning framework that enforces explicit length constraints over the entire trajectory. While more flexible, this method demands substantial training resources and still results in noticeable performance degradation compared to the original model.

Preprint.

We propose **Elastic Reasoning**, a simple yet effective method that enables large reasoning models to achieve scalable and adaptive length control. As illustrated in Figure 1, the S1 approach—generating the answer by emitting a special token such as "Final Answer"—performs better than directly truncating the full reasoning trajectory, underscoring the importance of preserving the solution segment. Motivated by this, we propose separate budgeting which explicitly divides the total token budget c into two parts: t tokens for the *thinking* phase and s tokens for the *solution* phase, where $c = t + s$. Once the model consumes t tokens in the thinking phase, we forcibly terminate it by appending the special token `</think>` and transition to solution generation. Separate budgeting outperforms S1 under varying generation budgets.

To further improve solution quality under incomplete reasoning, we introduce a novel training strategy called *budget-constrained rollout*, which teaches the model to generate high-quality answers even with partial CoT trajectories. This method is integrated into GRPO training and is highly efficient—requiring only 200 training steps on math tasks with a maximum response length of 2K tokens ($t^* = 1\text{K},\ s^* = 1\text{K}$), compared to 700 steps for L1-Exact and 820 steps for L1-Max with a 4K response length. Moreover, models trained with Elastic Reasoning generalize effectively to arbitrary reasoning budgets without the need for further fine-tuning.

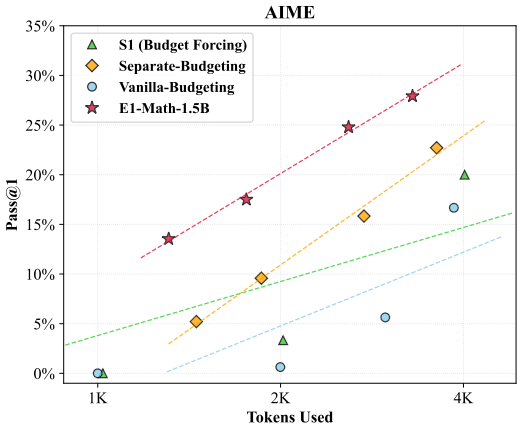


Figure 1: Separating thinking and solution phases enables better length control.

We evaluate Elastic Reasoning on both mathematical and programming reasoning tasks, introducing two models: **E1-Math-1.5B** and **E1-Code-14B**. (1) **E1-Math-1.5B** outperforms both L1-Exact and S1, and achieves performance comparable to L1-Max, while requiring significantly fewer training steps. For instance, on the AIME2024 dataset, our method achieves 35.0% accuracy, compared to 27.1% for L1-Max, 24.2% for L1-Exact, and 41.0% for the original model. (2) **E1-Code-14B** demonstrates strong scaling with varying inference budgets, achieving a Codeforces rating of 1987 and placing in the 96.0 percentile—comparable to O1-2024-12-17 (Low), which scores 1991 and ranks in the 96.1 percentile. (3) A surprising observation is that, after training, the trajectories generated by our models are significantly shorter than those from the original DeepScaleR and DeepCoder models across both math and code tasks. This suggests that budget-constrained rollout not only improves length control but also encourages the model to reason more concisely and generate more efficient solutions.

2 Related works

2.1 Test-time scaling in large language models

Increasing computation during inference, often referred to as test-time scaling (TTS), has been shown to improve the reasoning capabilities of LLMs [36, 35, 33, 4, 34, 24]. Early works, such as chain-of-thought prompting [36], show that producing a series of intermediate reasoning steps significantly improves LLMs' performance on complex reasoning tasks. Building on this, self-consistency [35] further boosts performance by sampling a diverse set of reasoning paths and selecting the most consistent answer. Recent studies have formalized these findings into test-time inference scaling laws [33, 37]. Wu et al. [37] explore the trade-offs between model size and inference-time computation. Snell et al. [33] investigated how fixed but non-trivial inference-time budgets can significantly boost LLM performance. The remarkable successes of advanced reasoning models, such as o1 [25] and R1 [4], have further amplified interest in leveraging TTS techniques. While much of the existing works primarily focuses on improving performance by increasing inference-time computation, our work takes a different perspective: *How can we enable LLMs to perform effective long reasoning under strict output length constraints?*

2.2 Length control in large language models

Controlling the generation length of an LLM directly affects both latency and monetary cost at inference time. Earlier approaches to length control are designed mainly for general text generation [13, 42]. Typical methods include (i) manipulating positional encodings to achieve exact sequence lengths [3], (ii) modifying training objectives to penalize deviations from length targets [13, 32], and (iii) fine-tuning on instructions that explicitly state the desired output length [42]. Although effective for tasks such as summarization or constrained writing, these techniques generally aim to verbosity or enforce maximum-length limits, and overlook the intricate, step-by-step reasoning processes required for many reasoning tasks. Recent works have begun to explore efficiency in reasoning by encouraging shorter chains [14, 2]; however, they typically lack mechanisms for precise, user-defined length targets that align with explicit compute budgets. One notable attempt, budget forcing [24], enforces strict token caps by truncating or padding with special tokens. This can yield incomplete reasoning or unnatural, forced outputs, ultimately harming both accuracy and interpretability. Additionally, L1 [1] uses reinforcement learning to let models dynamically allocate inference compute based on constraints provided in the prompt. Our approach does not need to include length instructions in the prompt. Instead, we truncate reasoning trajectories to meet a given budget and train the model under these constraints via reinforcement learning.

2.3 Efficient reasoning in large language models

Making complex reasoning in LLMs more efficient, particularly by shortening the reasoning process, is crucial to reducing computational costs and making these models practical for real-world deployment. This has become a vibrant research area with several promising directions to encourage more concise and effective reasoning strategies [14, 40, 10, 17, 19]. One common strategy involves incorporating explicit rewards into RL to encourage the model to find shorter reasoning paths [34, 19]. Some focus on creating datasets with examples of concise reasoning paths and then using SFT teach models how to generate compact and knowledgeable reasoning steps [14, 41]. Instead of relying solely on explicit textual reasoning, methods exploring latent reasoning aim to compress these intermediate steps into more compact, internal representations [10, 31, 28]. Efficiency can also be improved during inference, without needing to retrain the model. These training-free techniques dynamically adapt the reasoning strategy based on the specific input or task demands [17, 8]. In this work, we introduce a training approach using reinforcement learning under strict budget constraints to encourage the model to balance reasoning quality with cost efficiency.

3 Methodology

3.1 Preliminaries: Reasoning Language Models

We consider reasoning-augmented language models that generate outputs consisting of two distinct segments: a *thinking* part and a *solution* part. Following prior work, we denote the reasoning phase using special tokens such as `<think>` and `</think>` to explicitly mark the model's intermediate thoughts.

Formally, given an input prompt x, the model generates an output sequence $y = (y^{\text{think}}, y^{\text{solution}})$, where y^{think} contains the intermediate reasoning steps (enclosed between `<think>` and `</think>`) and y^{solution} contains the final solution. Typically, y^{think} accounts for over 90% of the total tokens, while y^{solution} provides a concise summary and final answer. The overall generation structure is:

$$y = (\texttt{<think>}\ \text{intermediate reasoning}\ \texttt{</think>},\ \text{solution})$$

3.2 Elastic reasoning

3.2.1 Budget-constrained inference

In many real-world applications, inference cost must be carefully controlled due to constraints on latency, computation, or memory. A common approach is to truncate generation after a fixed number of tokens c, enforcing:

$$|y| \leq c$$

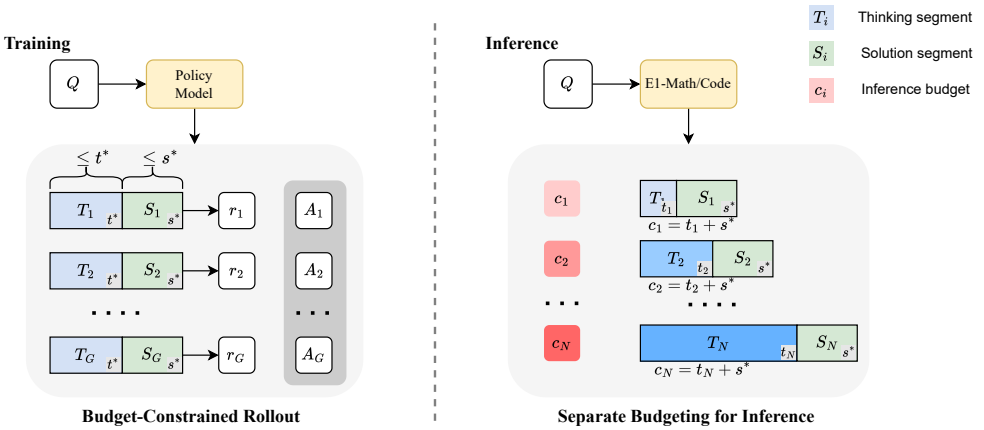


Figure 2: The framework of Elastic Reasoning. Elastic Reasoning comprises two key components: (1) GRPO training with budget-constrained rollout, and (2) separate budgeting for inference. **Left:** During training, the model is optimized using GRPO under a fixed token budget (t^*, s^*). **Right:** At inference time, the trained E1 model can generalize to arbitrary token budgets $c_i = t_i + s^*$, enabling flexible and efficient reasoning.

where $|y|$ denotes the number of generated tokens. However, naively truncating the output often results in incomplete or missing y^{solution}, leading to invalid or unusable predictions.

3.2.2 Separate budgeting for thinking and solution

To address this limitation, we propose *Separate Budgeting*, a method that explicitly allocates independent budgets for the reasoning and solution phases. A key observation is that even when the reasoning phase is forcibly terminated (e.g., by inserting `</think>`), the model is still capable of producing a coherent—and often correct—solution.

Given a total generation budget c, we divide it into two components: a budget t for the thinking phase and a budget s for the solution phase, such that $c = t + s$.

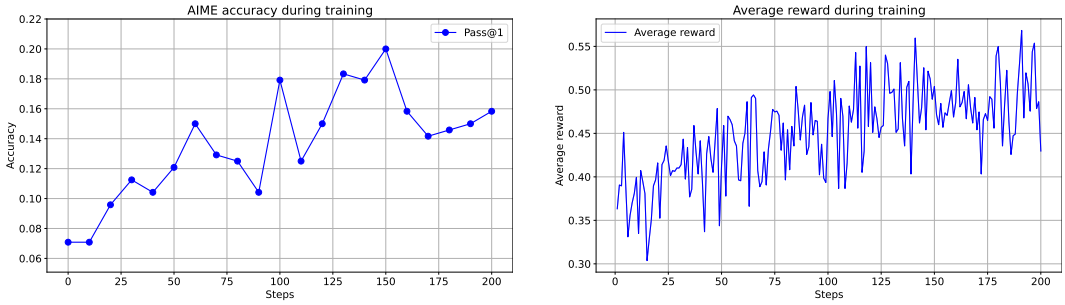
During inference:

- The model begins generating within a `<think>` block.
- If the model emits `</think>` before reaching the budget t, we transition immediately to the solution phase.
- If the budget t is exhausted before `</think>` is emitted, we forcibly terminate the reasoning by appending `</think>`.
- The model then continues generating the solution segment, up to a maximum of s tokens.

This approach ensures that both the reasoning and solution components are explicitly accommodated within the total budget c, thereby avoiding unintended truncation of the solution segment. The thinking budget t can be flexibly adjusted at inference time to match different application scenarios, while the solution phase always retains a guaranteed allocation. As shown in Figure 1, Separate Budgeting outperforms both vanilla budgeting (naïve truncation) and S1 (budget forcing). By dedicating a fixed token budget for solution generation, Separate Budgeting significantly improves the reliability and quality of model outputs under tight inference-time constraints.

3.2.3 Budget-constrained rollout

While Separate Budgeting ensures dedicated budgets for both reasoning and solution phases, we observe that naively truncating the thinking part—especially on complex tasks such as code generation—can lead to significant performance degradation. To mitigate this issue, we propose a reinforcement learning (RL) fine-tuning procedure that explicitly trains the model under reason-



(a) Pass@1 on AIME2024 over steps

(b) Reward over steps

Figure 3: Validation accuracy and reward curves of E1-Math-1.5B over training steps.

ing budget constraints, allowing it to produce more effective and concise reasoning within limited budgets.

We adopt GRPO as our RL algorithm. Let π_θ denote the policy of a language model parameterized by θ, which generates a response $y = (y^{\text{think}}, y^{\text{solution}})$ for a given input x, subject to a total budget constraint $t^* + s^* = c^*$. During training, we simulate the Separate Budgeting procedure used at inference time: the policy rolls out a reasoning segment y^{think} up to a maximum of t^* tokens. If the model emits the `</think>` token before reaching this limit, it proceeds to generate the solution segment as usual. Otherwise, we forcibly append `</think>` once the budget t^* is reached. The model then generates the solution segment y^{solution} using the remaining s^* tokens.

Let $r(y)$ denote a task-specific reward function. The training objective is to maximize the expected reward:

$$J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x;\ t^*,s^*)}\left[r(y)\right]$$

We optimize $J(\theta)$ using GRPO with the following gradient estimator:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x,y}\left[A(x,y)\nabla_\theta \log \pi_\theta(y \mid x;\ t^*, s^*)\right], \quad A(x,y) = \frac{r(y) - \mathbb{E}_{y'\sim\pi_\theta(\cdot|x;\ t^*,s^*)}[r(y')]}{\sqrt{\mathbb{V}_{y'\sim\pi_\theta(\cdot|x)}[r(y')]}}$$

In our training setup, we fix the budget pair to $(t^*, s^*) = (1\text{K}, 1\text{K})$ for simplicity and efficiency. Surprisingly, we find that the learned policy generalizes well to a wide range of unseen budget configurations at test time, without requiring any additional fine-tuning. As shown in Figure 1, the E1-Math-1.5B model achieves substantial improvements while generalizing robustly across various generation budgets. This indicates that Elastic Reasoning encourages the model to internalize a flexible reasoning strategy that adapts to different resource constraints.

This RL-based adaptation helps the model prioritize informative reasoning content earlier in the generation process, thereby improving both robustness and solution quality under test-time truncation.

4 Experiment results

4.1 Models and datasets

Our base models are DeepScaleR-1.5B-Preview [21] and DeepCoder-14B-Preview [20], which are fine-tuned from DeepSeekR1-Distill-Qwen-1.5B and 14B [4] through iterative context lengthening. For training data, we follow the same datasets used in [21, 20]. In the math domain, the training set consists of AIME (1984-2023), AMC, Omni-Math [9], and STILL [23]. For code training, we use TACO [16], SYNTHETIC-1 [22], and LiveCodeBench (2023/05/01-2024/07/31) [12]. For evaluation, we use AIME 2024, MATH500 [11], AMC, Olympiad-Bench [9], and Minerva Math [15] for mathematical reasoning. For code-related tasks, we evaluate on LiveCodeBench (2024/08/01-2025/02/01) [12], Codeforces, and HumanEval+ [18]. More training details are in Appendix B.

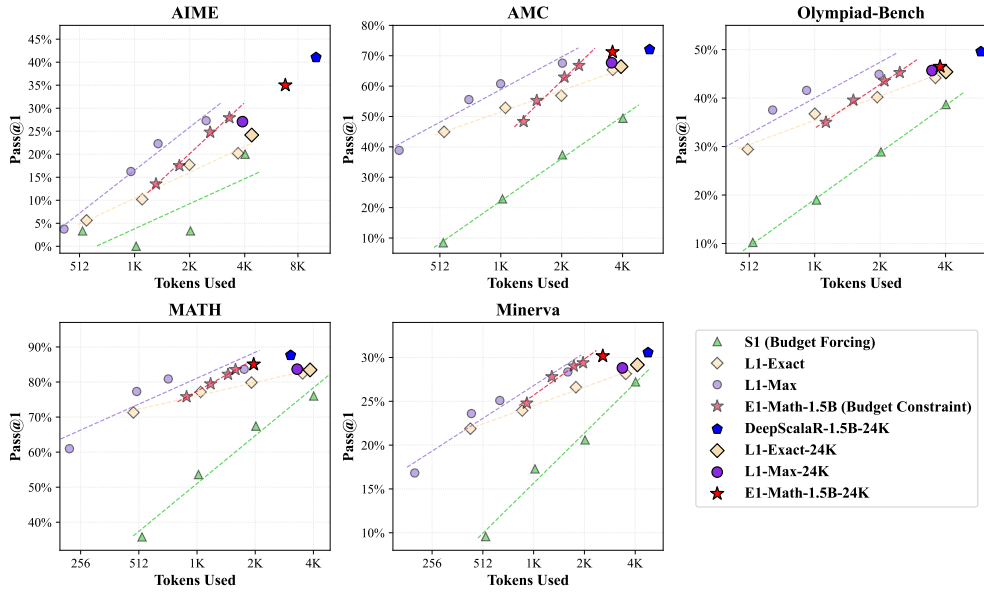


Figure 4: Comparison of E1-Math-1.5B with L1 and S1 baselines under varying generation budgets.

4.2 Mathematical reasoning rsults

We visualize the reward and validation Pass@1 performance on AIME2024 every 10 steps during training in Figure 3. It can be observed that the reward steadily increases during the initial training phase and begins to converge after approximately the 150$^{\text{th}}$ step. Meanwhile, the validation accuracy (Pass@1) improves rapidly, rising from around 0.07 to 0.20 over the course of training. This demonstrates that, through budget-constrained rollout, the model can quickly learn to reason effectively when the thinking phase is incomplete.

We report Pass@1 accuracy versus the number of tokens used across five math benchmarks AIME, AMC, Olympiad-Bench, MATH500, and Minerva Math in Figure 4. Our proposed method, E1-Math-1.5B, under both budget-constrained and 24K-token settings (red stars), consistently outperforms S1 (Budget Forcing) and L1-Exact, and performs competitively with L1-Max, while requiring significantly fewer training steps. On MATH500, E1-Math-1.5B achieves a Pass@1 accuracy of 83.6% using only 1619 tokens per question, whereas L1-Exact and L1-Max yield lower or comparable performance with more tokens (L1-Exact: 79.9% with 1959 tokens; L1-Max: 83.6% with 1796 tokens). Notably, when evaluated without inference-time budget constraints, E1-Math-1.5B achieves higher accuracy than all baseline methods across all benchmarks. For example, on AIME2024, E1-Math-1.5B exhibits a performance degradation of only 6.0% relative to the original model, compared to 12.9% for L1-Max and 16.8% for L1-Exact. These results demonstrate that our method is not only effective in enforcing inference-time budget constraints but also preserves most of the original model's performance. When compared with the original DeepScaleR-1.5B, E1-Math-1.5B reduces the average number of tokens used across datasets by more than 30%, including a 32.1% reduction on AIME2024.

Furthermore, similar to L1, S1, and O1, we observe a clear log-linear scaling pattern in E1: performance improves approximately linearly with respect to the logarithm of the number of generated reasoning tokens.

4.3 Code reasoning results

As shown in Figure 5, we visualize the Pass@1 accuracy on LiveCodeBench under varying generation budgets, comparing our method to a simple separate budgeting strategy for thinking and solution. We observe that the original **DeepCoder-14B-Preview** fails to generate correct outputs when reasoning is incomplete, consistently achieving less than 10% accuracy when inference budget is less than 4K even using separate budgeting. In contrast, our E1-Code-14B model demonstrates impressive scalability:

Table 1: Comparison of models across LiveCodeBench, Codeforces, HumanEval+, and AIME benchmarks. E1-code-14B variants trained exclusively on code data; their AIME scores, obtained on math problems unseen during training, demonstrate that E1-code-14B retains strong math performance.

Model	LiveCodeBench	Codeforces Rating	Codeforces Percentile	HumanEval+	AIME
O1-2024-12-17 (Low)	59.5	**1991**	**96.1**	90.8	**74.4**
O3-Mini-2025-1-31 (Low)	**60.9**	1918	94.9	**92.6**	60.0
O1-Preview	42.7	1658	88.5	89.0	40.0
DeepSeek-R1	**62.8**	1948	95.4	**92.6**	**79.8**
DeepSeek-R1-Distill-Qwen-14B	53.0	1791	92.7	92.0	69.7
DeepCoder-14B-Preview[1]	58.1	1945	95.4	90.8	71.7
E1-code-14B ($t = 1k, a = 1k$)	37.3	1457	78.1	88.3	17.9
E1-code-14B ($t = 2k, a = 1k$)	41.6	1604	85.4	89.6	28.5
E1-code-14B ($t = 3k, a = 1k$)	44.1	1711	90.6	90.8	35.4
E1-code-14B ($t = 4k, a = 1k$)	47.0	1771	92.3	92.0	41.9
E1-code-14B	$58.4_{+0.3}$	$\mathbf{1987}_{+42}$	$\mathbf{96.0}_{+0.6}$	$91.4_{+0.6}$	70.6

its performance improves steadily as the inference budget increases, highlighting the effectiveness of our training strategy in enabling the model to reason adaptively under constrained thinking. Notably, E1-Code-14B also achieves a performance improvement of 0.3% on LiveCodeBench even in the unconstrained setting, while simultaneously reducing the average number of generated tokens by **37.4%**—from 17,815 to 11,145 tokens. This indicates that our method not only scales well with inference budgets but also promotes more concise and efficient reasoning.

In Table 1, we report the performance of E1-Code-14B on four benchmarks: LiveCodeBench, Codeforces, HumanEval Plus, and AIME2024. We observe consistent test-time scaling behavior across all benchmarks under constrained inference budgets. Beyond scalability, our model also demonstrates strong performance in the unconstrained setting. Specifically, we observe performance improvements on LiveCodeBench, Codeforces, and HumanEval Plus, and only a slight performance drop on AIME2024. On Codeforces, E1-Code-14B achieves a 42-point improvement in rating and a 0.6 percentile gain, outperforming O3-Mini-2025-1-31 (Low) and performing comparably to O1-2024-12-17 (Low). These results highlight that our method not only enables efficient, budget-constrained reasoning but also enhances overall reasoning capability, even in unconstrained scenarios.

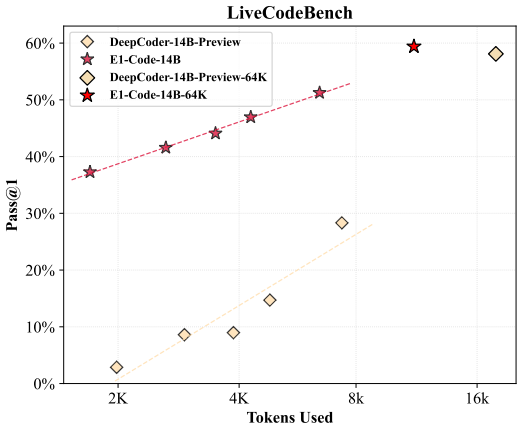


Figure 5: Pass@1 accuracy on LiveCodeBench under varying reasoning budgets. Both of the models inference with separate budgeting.

4.4 Analysis and discussions

4.4.1 Which part is enhanced after training?

To better understand which components of the reasoning process are enhanced through training, we conduct ablation experiments on DeepScaleR-1.5B-Preview and E1-Math-1.5B using the AIME2024 benchmark. Specifically, we separately generate the *thinking* and *solution* segments using both models under varying generation budgets. For example, we use DeepScaleR-1.5B-Preview to generate the thinking part, and then use E1-Math-1.5B to generate the corresponding solution based on that reasoning. This setup allows us to isolate the contributions of each model to the reasoning pipeline and assess how training improves each component.

As shown in Table 2, we observe that both the thinking and solution are enhanced after training. Notably, the improvement in the solution component is more substantial, particularly under constrained thinking budgets. For instance, using the E1 model to generate only the solution segment yields

[1]Results are reproduced using the authors' official code and model with the same evaluation protocol.

Table 2: Ablation of enhanced thinking and solution on DeepScaleR-1.5B-Preview and E1-Math-1.5. Budget is in format 'thinking+solution' (in thousands of tokens).

DeepScaleR-1.5B		E1-Math-1.5B		Pass@1 (%)			
Thinking	Solution	Thinking	Solution	0.5K+1K	1K+1K	2K+1K	3K+1K
✓	✓			2.10	4.80	12.5	20.0
	✓	✓		$3.50_{+1.4}$	$7.90_{+3.1}$	$20.6_{+8.1}$	$24.0_{+4.0}$
✓			✓	$10.8_{+8.7}$	$14.2_{+9.4}$	$21.9_{+9.4}$	$26.4_{+6.4}$
		✓	✓	$13.5_{+\mathbf{11.4}}$	$17.5_{+\mathbf{12.7}}$	$24.8_{+\mathbf{12.3}}$	$27.9_{+\mathbf{7.9}}$

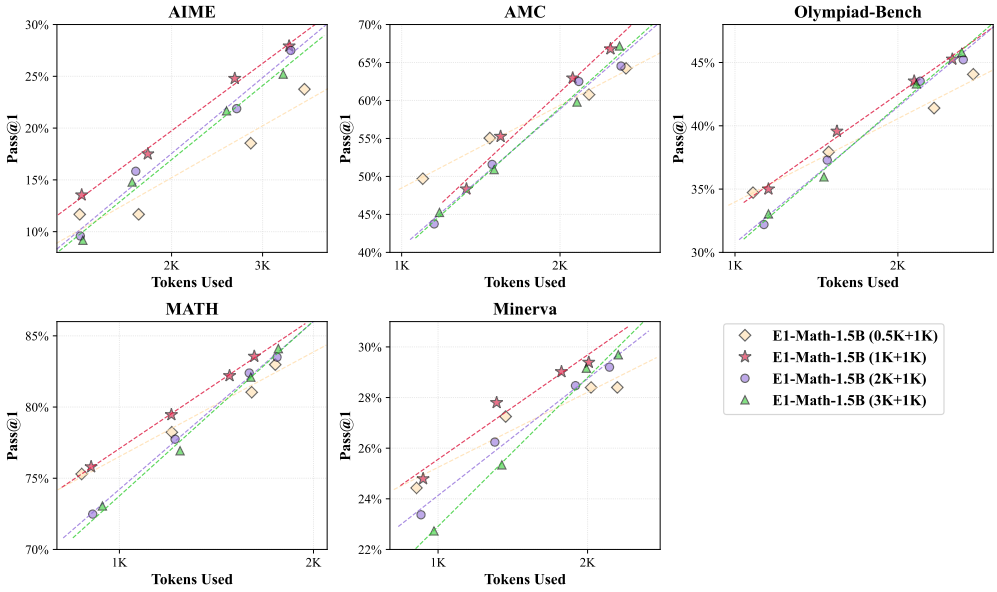


Figure 6: Ablation study on training reasoning budget t^*. We compare four settings: $t^* \in \{0.5\text{K}, 1\text{K}, 2\text{K}, 3\text{K}\}$, while keeping the solution budget fixed at $a^* = 1\text{K}$.

an 8.7% gain in accuracy compared to using the original DeepScaleR model, under a generation budget of $(0.5\text{K} + 1\text{K})$ tokens. This highlights the effectiveness of budget-constrained rollout in strengthening the model's ability to produce high-quality solutions based on incomplete reasoning.

This observation also helps explain why training with a fixed budget constraint (e.g., $(1\text{K}, 1\text{K})$) enables the model to generalize effectively to a wide range of budget configurations. We hypothesize that the improvement in solution generation plays a central role in this generalization, allowing the model to adapt even when the available thinking tokens are reduced.

4.4.2 Ablation of training budget t^*

To further investigate the role of the thinking budget t^* in our proposed budget-constrained rollout, we conduct experiments to evaluate the model's performance under four settings: $t^* \in \{0.5\text{K}, 1\text{K}, 2\text{K}, 3\text{K}\}$, while keeping the solution budget fixed at $a^* = 1\text{K}$. We evaluate on five math benchmarks: AIME, AMC, Olympiad-Bench, MATH500, and Minerva Math (Figure 6).

Across all configurations, the model demonstrates strong generalization to varying inference budgets on all benchmarks. Among the tested values, $t^* = 1\text{K}$ consistently achieves the best performance, while also maintaining a low maximum generation length of 2K tokens, making it a highly efficient and effective setting. Based on this trade-off between performance and computational cost, we adopt $(t^* = 1\text{K},\ s^* = 1\text{K})$ as our default configuration.

4.4.3 Token allocation between thinking and solution

Figure 7 visualizes the distribution of *thinking* and *solution* tokens within generated trajectories under different generation budget constraints. We select AIME2024 for the math task and LiveCodeBench for the coding task.

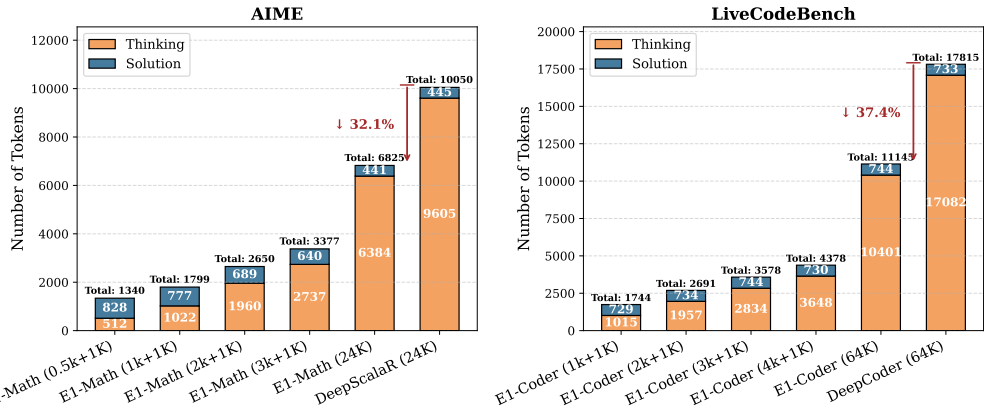


Figure 7: Distribution of tokens for thinking and solution across different generation budgets.

For AIME2024, as the inference budget decreases, the number of tokens used in the thinking segment decreases accordingly, while the number of tokens in the solution segment slightly increases. A similar trend is observed on LiveCodeBench, where the thinking tokens decrease with tighter budgets, while the number of solution tokens remains relatively stable.

Notably, even when evaluated without budget constraints, our trained E1 models demonstrate substantial token efficiency: they reduce total token usage by 32.1% on AIME2024 and 37.4% on LiveCodeBench, while maintaining strong performance (even slightly better than the baseline model). This suggests that the model has learned to reason more concisely and generate efficient solutions post training.

4.4.4 Iterative training

Table 3: Pass@1(%) on AIME 2024 across different budget configurations in two iterations.

Iteration	0.5K+1K	1K+1K	2K+1K	3K+1K
1st	13.5	17.5	24.8	27.9
2nd	11.5	17.1	22.9	26.7

In this section, we investigate whether the model can benefit from *iterative training*—that is, performing a second round of training with expanded budgets after initial training. Specifically, we first train the model using a budget of $(t^* = 1\text{K},\ a^* = 1\text{K})$, and then continue training from the resulting checkpoint using a larger thinking budget of $(t^* = 3\text{K},\ a^* = 1\text{K})$. The results on AIME2024 are reported in Table 3.

Surprisingly, the second round of training does not improve performance. In fact, we observe a slight degradation in accuracy, suggesting that once the model has learned to reason effectively under a shorter budget, further training with a longer budget may not provide additional benefits.

5 Conclusion

We introduce **Elastic Reasoning**, a unified framework for enabling large reasoning models to generate accurate and efficient chain-of-thought outputs under strict inference-time constraints. By explicitly separating the reasoning process into *thinking* and *solution* phases, and training with a novel *budget-constrained rollout* strategy, our approach ensures robustness to truncated reasoning while preserving or even improving overall performance. Elastic Reasoning significantly reduces token usage during inference, generalizes across unseen budget configurations, and outperforms prior length control baselines in both mathematical and programming domains. Our findings offer a scalable and principled solution for real-world deployment of reasoning LLMs where computation budgets are limited. We believe this framework opens new directions for budget-aware reasoning.

References

[1] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. URL 2025.

[2] Daman Arora and Andrea Zanette. Training language models to reason efficiently, 2025. URL https://arxiv.org/abs/2502.04463.

[3] Bradley Butcher, Michael O'Keefe, and James Titchener. Precise length control in large language models, 2024. URL https://arxiv.org/abs/2412.11937.

[4] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

[5] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. Raft: Reward ranked finetuning for generative foundation model alignment. *arXiv preprint arXiv:2304.06767*, 2023.

[6] Hanze Dong, Wei Xiong, Bo Pang, Haoxiang Wang, Han Zhao, Yingbo Zhou, Nan Jiang, Doyen Sahoo, Caiming Xiong, and Tong Zhang. Rlhf workflow: From reward modeling to online rlhf. *arXiv preprint arXiv:2405.07863*, 2024.

[7] Yifan Du, Zikang Liu, Yifan Li, Wayne Xin Zhao, Yuqi Huo, Bingning Wang, Weipeng Chen, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. Virgo: A preliminary exploration on reproducing o1-like mllm. *arXiv preprint arXiv:2501.01904*, 2025.

[8] Yichao Fu, Junda Chen, Yonghao Zhuang, Zheyu Fu, Ion Stoica, and Hao Zhang. Reasoning without self-doubt: More efficient chain-of-thought through certainty probing. In *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025.

[9] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-math: A universal olympiad level mathematic benchmark for large language models, 2024. URL https://arxiv.org/abs/2410.07985.

[10] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

[11] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL https://arxiv.org/abs/2103.03874.

[12] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL arXivpreprintarXiv:2403.07974.

[13] Renlong Jie, Xiaojun Meng, Lifeng Shang, Xin Jiang, and Qun Liu. Prompt-based length controlled generation with reinforcement learning. *arXiv preprint arXiv:2308.12030*, 2023.

[14] Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness, 2024. URL https://arxiv.org/abs/2412.11664.

[15] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models, 2022. URL arXivpreprintarXiv:2206.14858.

[16] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. Taco: Topics in algorithmic code generation dataset, 2023. URL arXivpreprintarXiv:2312.14852.

[17] Baohao Liao, Yuhui Xu, Hanze Dong, Junnan Li, Christof Monz, Silvio Savarese, Doyen Sahoo, and Caiming Xiong. Reward-guided speculative decoding for efficient llm reasoning. *arXiv preprint arXiv:2501.19324*, 2025.

[18] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation, 2023. URL https://openreview.net/forum?id=1qvx610Cu7.

[19] Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. O1-pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. *arXiv preprint arXiv:2501.12570*, 2025.

[20] Michael Luo, Xiaoxiang Shi Sijun Tan, Roy Huang, Rachel Xin, Colin Cai, Ameen Patel, Alpay Ariyak, Qingyang Wu, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepcoder: A fully open-source 14b coder at o3-mini level. https://pretty-radio-b75.notion.site/DeepCoder-A-Fully-Open-Source-14B-Coder-at-O3-mini-Level-1cf81902c14680b3bee5eb349a512a51, 2025. Notion Blog.

[21] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, et al. Deepscaler: Surpassing O1-preview with a 1.5 b model by scaling reinforcement learning, 2025. URL https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013 Notion blog post.

[22] Justus Mattern, Sami Jaghouar, Manveer Basra, Jannik Straube, Matthew Di Ferrante, Felix Gabriel, Jack Min Ong, Vincent Weisser, and Johannes Hagemann. Synthetic-1: Two million collaboratively generated reasoning traces from deepseek-r1, 2025. URL https://www.primeintellect.ai/blog/synthetic-1-release.

[23] Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, Yiwen Hu, Yiru Tang, Jiapeng Wang, Xiaoxue Cheng, Huatong Song, Wayne Xin Zhao, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems, 2024. URL https://arxiv.org/abs/2412.09413.

[24] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL https://arxiv.org/abs/2501.19393.

[25] OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alex Tachard Passos, Alexander Neitz, Alexander Prokofiev, Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Eric Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, Fred von Lohmann, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian Osband, Ignasi Clavera Gilaberte, Ilge Akkaya, Ilya Kostrikov, Ilya Sutskever, Irina Kofman, Jakub Pachocki, James Lennon, Jason Wei, Jean Harb, Jerry Twore, Jiacheng Feng, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joaquin Quiñonero Candela, Joe Palermo, Joel Parish, Johannes Heidecke, John Hallman, John Rizzo, Jonathan Gordon, Jonathan Uesato, Jonathan Ward, Joost Huizinga, Julie Wang, Kai Chen, Kai Xiao, Karan Singhal, Karina Nguyen, Karl Cobbe, Katy Shi, Kayla Wood, Kendra Rimbach, Keren Gu-Lemberg, Kevin Liu, Kevin Lu, Kevin Stone, Kevin Yu, Lama Ahmad, Lauren Yang, Leo Liu, Leon Maksin, Leyton Ho, Liam Fedus, Lilian Weng, Linden Li, Lindsay McCallum, Lindsey Held, Lorenz Kuhn, Lukas Kondraciuk, Lukasz Kaiser, Luke Metz, Madelaine Boyd, Maja Trebacz, Manas Joglekar, Mark Chen, Marko Tintor, Mason Meyer, Matt Jones, Matt Kaufer, Max Schwarzer, Meghan Shah, Mehmet Yatbaz, Melody Y. Guan, Mengyuan Xu, Mengyuan Yan, Mia Glaese, Mianna Chen, Michael Lampe, Michael Malek, Michele Wang, Michelle Fradin, Mike McClay, Mikhail Pavlov, Miles Wang, Mingxuan Wang, Mira Murati, Mo Bavarian, Mostafa Rohaninejad, Nat McAleese, Neil Chowdhury, Neil Chowdhury, Nick Ryder, Nikolas Tezak, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk, Olivia Watkins, Patrick Chao, Paul Ashbourne, Pavel Izmailov, Peter Zhokhov, Rachel Dias, Rahul Arora, Randall Lin, Rapha Gontijo Lopes, Raz Gaon, Reah Miyara, Reimar Leike, Renny Hwang, Rhythm Garg, Robin Brown, Roshan James, Rui Shu, Ryan Cheu, Ryan Greene, Saachi Jain, Sam Altman, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Santiago Hernandez, Sasha Baker, Scott McKinney, Scottie Yan, Shengjia Zhao, Shengli Hu, Shibani Santurkar, Shraman Ray Chaudhuri, Shuyuan Zhang, Siyuan Fu, Spencer Papay, Steph Lin, Suchir Balaji, Suvansh Sanjeev, Szymon Sidor, Tal Broda, Aidan Clark, Tao Wang, Taylor Gordon, Ted Sanders, Tejal Patwardhan, Thibault Sottiaux, Thomas Degry, Thomas Dimson, Tianhao Zheng, Timur Garipov, Tom Stasi, Trapit Bansal, Trevor Creech, Troy Peterson, Tyna Eloundou, Valerie Qi, Vineet Kosaraju, Vinnie Monaco, Vitchyr Pong, Vlad Fomenko, Weiyi Zheng, Wenda Zhou, Wes McCabe, Wojciech Zaremba, Yann Dubois, Yinghai Lu, Yining Chen, Young Cha, Yu Bai, Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. Openai o1 system card, 2024. URL https://arxiv.org/abs/2412.16720.

[26] Yiwei Qin, Xuefeng Li, Haoyang Zou, Yixiu Liu, Shijie Xia, Zhen Huang, Yixin Ye, Weizhe Yuan, Hector Liu, Yuanzhi Li, et al. O1 replication journey: A strategic progress report–part 1. *arXiv preprint arXiv:2410.18982*, 2024.

[27] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36:53728–53741, 2023.

[28] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J Reddi. Reasoning with latent thoughts: On the power of looped transformers. *arXiv preprint arXiv:2502.17416*, 2025.

[29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[30] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[31] Xuan Shen, Yizhou Wang, Xiangxi Shi, Yanzhi Wang, Pu Zhao, and Jiuxiang Gu. Efficient reasoning with hidden thinking. *arXiv preprint arXiv:2501.19201*, 2025.

[32] Prasann Singhal, Tanya Goyal, Jiacheng Xu, and Greg Durrett. A long way to go: Investigating length correlations in rlhf, 2024. URL https://arxiv.org/abs/2310.03716.

[33] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL https://arxiv.org/abs/2408.03314.

[34] Kimi Team, A Du, B Gao, B Xing, C Jiang, C Chen, C Li, C Xiao, C Du, C Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms, 2025. URL https://arxiv.org/abs/2501.12599.

[35] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL https://arxiv.org/abs/2203.11171.

[36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

[37] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models, 2024. URL https://arxiv.org/abs/2408.00724.

[38] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, et al. A minimalist approach to llm reasoning: from rejection sampling to reinforce. *arXiv preprint arXiv:2504.11343*, 2025.

[39] Wei Xiong, Hanning Zhang, Chenlu Ye, Lichang Chen, Nan Jiang, and Tong Zhang. Self-rewarding correction for mathematical reasoning. *arXiv preprint arXiv:2502.19613*, 2025.

[40] Yuhui Xu, Zhanming Jie, Hanze Dong, Lei Wang, Xudong Lu, Aojun Zhou, Amrita Saha, Caiming Xiong, and Doyen Sahoo. Think: Thinner key cache by query-driven pruning. *arXiv preprint arXiv:2407.21018*, 2024.

[41] Ping Yu, Jing Xu, Jason Weston, and Ilia Kulikov. Distilling system 2 into system 1. *arXiv preprint arXiv:2407.06023*, 2024.

[42] Weizhe Yuan, Ilia Kulikov, Ping Yu, Kyunghyun Cho, Sainbayar Sukhbaatar, Jason Weston, and Jing Xu. Following length constraints in instructions, 2024. URL https://arxiv.org/abs/2406.17744.

[43] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

[44] Yuxiang Zhang, Shangxi Wu, Yuqi Yang, Jiangming Shu, Jinlin Xiao, Chao Kong, and Jitao Sang. o1-coder: an o1 replication for coding. *arXiv preprint arXiv:2412.00154*, 2024.

A Limitations and broader impacts

While Elastic Reasoning offers a practical and scalable solution for length-controllable inference, it introduces a few limitations. First, our method assumes that the reasoning process can be meaningfully segmented into distinct thinking and solution phases, which may not hold for tasks with highly interleaved reasoning and answering (e.g., some forms of dialogue or commonsense reasoning). Second, our evaluation focuses primarily on math and code reasoning tasks; further investigation is needed to assess the effectiveness of Elastic Reasoning in domains such as scientific question answering or multi-hop retrieval.

Elastic Reasoning aims to improve the deployability and controllability of large reasoning models by enabling efficient inference under strict compute constraints. This can have positive downstream effects, such as reducing energy consumption, lowering latency for interactive applications, and enabling access to reasoning-capable models in resource-limited environments. At the same time, the ability to tailor model outputs based on budget constraints introduces new challenges: models may be optimized to generate shorter outputs without transparent communication of omitted reasoning, potentially affecting interpretability or user trust. Additionally, as reasoning models are increasingly used in high-stakes domains (e.g., education, law, healthcare), the separation between "thinking" and "solution" phases may require careful design to ensure that important justifications are not lost during inference-time truncation. We encourage future work to explore human-centered evaluations of controllable reasoning, and to develop standards for safely deploying budget-aware models in real-world applications.

B Training details

For GRPO training, we adopt the same hyperparameters as those used in DeepScaleR-1.5B-Preview and DeepCode-14B-Preview. For E1-Math-1.5B, we use a learning rate of 1×10^{-6} and a batch size of 128. The maximum context length is set to 1K tokens for the thinking segment and 1K tokens for the solution segment. Training is performed for 200 steps using the `VeRL` [**?**] framework. For E1-Code-14B, we use the same learning rate and batch size. The context length configuration mirrors that of the math model: 1K tokens for thinking and 1K tokens for solution. Training is conducted for only 30 steps.